

Studiu Comparativ al Algoritmilor de Sortare

David Tudor

Universitatea de Vest din Timișoara

Facultatea de Informatică

david.tudor06@e-uvv.ro

Mai 2026

Rezumat

Acest proiect analizează în mod practic cum funcționează cinci algoritmi de sortare de bază. Mai exact, am vrut să văd cum modul în care sunt așezate datele inițial și cantitatea lor ajung să afecteze viteza reală de procesare pe calculator.

Pentru a înțelege mai bine rezultatele, am împărțit testele în segmente mai mici de date.

Cuprins

1	Introducere	2
1.1	Context general	2
1.2	Motivația proiectului	2
1.3	Obiectivele lucrării	2
2	Evoluția, Funcționarea și Istoricul Algoritmilor	2
2.1	Bubble Sort (Sortarea prin Metoda Bulelor)	2
2.2	Selection Sort (Sortarea prin Selecție)	3
2.3	Insertion Sort (Sortarea prin Inserție)	3
2.4	Merge Sort (Sortarea prin Interclasare)	3
2.5	Quick Sort (Sortarea Rapidă)	4
3	Arhitectura Proiectului	4
4	Rezultate și Analiză Empirică	4
5	Evaluarea Scalabilității: Analiză Grafică	5
5.1	Fereastra Micro: 10 - 250 elemente	5
5.2	Fereastra Small: 250 - 500 elemente	5
5.3	Fereastra Medium: 1.000 - 5.000 elemente	6
5.4	Fereastra Large: 5.000 - 20.000 elemente	6
6	Investigarea Anomaliilor de Performanță	6
7	Discuție Comparativă cu Literatura de Specialitate	7
8	Concluzii	7

1 Introducere

1.1 Context general

În era digitală actuală, gestionarea și organizarea eficientă a volumelor mari de date reprezintă o necesitate critică. Algoritmii de sortare constituie piloni fundamentali în optimizarea bazelor de date, a motoarelor de căutare și a sistemelor de operare, unde reducerea timpului de execuție și economisirea resurselor hardware sunt esențiale.

1.2 Motivația proiectului

Motivația acestei lucrări rezidă în analiza decalajului dintre complexitatea teoretică a algoritmilor (notațiile $O(n^2)$ sau $O(n \log n)$) [2] și comportamentul lor practic (empiric). Pe sistemele moderne, performanța reală a codului este influențată critic de factori hardware și de starea inițială a setului de date (ordonat, inversat sau aleatoriu).

1.3 Obiectivele lucrării

Pentru a evalua aceste aspecte, proiectul își propune următoarele obiective:

- Dezvoltarea unui instrument de testare (*benchmark*) precis în C++.
- Evaluarea comparativă a 5 algoritmi reprezentativi (*Bubble*, *Selection*, *Insertion*, *Merge*, *Quick Sort*).
- Analiza performanței pe 4 distribuții diferite de date, evidențiind avantajele și anomaliiile fiecărei metode de sortare.

2 Evoluția, Funcționarea și Istoricul Algoritmilor

Fiecare algoritm analizat poartă amprenta epocii sale și a limitărilor tehnologice pe care inventatorii săi au încercat să le depășească.

În sine apariția algoritmilor a fost forțată de necesitatea organizării eficiente a datelor, pornind de la metode simple intuitive la cele complexe din zilele noastre aplicate pe calculatoarele moderne.

2.1 Bubble Sort (Sortarea prin Metoda Bulelor)

Context istoric: A fost descris formal în 1956 de Edward H. Friend [9]. Deși azi îl folosim mai mult la școală ca să învățăm bazele programării, el ne arată foarte clar de ce nu este bine să pui calculatorul să facă pași inutili. La un număr mare de date, algoritmul pur și simplu durează prea mult.

Cum funcționează: Algoritmul ia lista de la un capăt la altul și compară mereu câte două numere vecine. Dacă stânga e mai mare ca dreapta, le schimbă între ele. Repetă treaba asta până când lista e complet ordonată. Numele vine de la faptul că numerele mari „se ridică” încet spre finalul listei, în același mod cum bulele de aer ies la suprafață.

Proprietate	Detalii
Complexitate teoretică	Timp: $O(n)$ (optim), $O(n^2)$ (mediu/worst). Memorie: $O(1)$.
Stabilitate	Stabil.
Scenariul ideal	Liste deja sortate (dacă se verifică absența schimbărilor).
Scenariul defavorabil	Datele sunt ordonate descrescător (invers).

2.2 Selection Sort (Sortarea prin Selecție)

Context istoric: A apărut undeva prin anii '50 [1]. Pe atunci, salvarea datelor pe memoria calculatoarelor vechi era un chin: dura mult și strica componentele fizice. Selection Sort a fost inventat specific ca numerele să fie mutate de cât mai puține ori.

Cum funcționează: Ideea e destul de simplă. Caută cel mai mic număr din toată lista și îl pune pe prima poziție. Apoi caută al doilea cel mai mic număr și îl mută pe a doua poziție, și tot așa până lista este sortată.

Proprietate	Detalii
Complexitate teoretică	Timp: $O(n^2)$ în toate cazurile. Memorie: $O(1)$.
Stabilitate	Instabil (de regulă).
Scenariul ideal	Nu există un caz neapărat ideal, algoritmul este constant, dar în același timp este și mai lent.

2.3 Insertion Sort (Sortarea prin Inserție)

Context istoric: A fost gândit de John Mauchly în 1946 [1] pentru uriașul calculator ENIAC¹. Deoarece urmează o logică atât de naturală, am observat în testele mele că este incredibil de rapid dacă îi dai o listă care este deja aranjată pe jumătate.

Cum funcționează: Gândiți-vă la cum vă așezați cărțile de joc în mână. Iei o carte nouă și o bagi exact la locul ei, printre cele deja așezate. Fix așa procedează și Insertion Sort.

Proprietate	Detalii
Complexitate teoretică	Timp: $O(n)$ (optim), $O(n^2)$ (worst). Memorie: $O(1)$.
Stabilitate	Stabil.
Scenariul ideal	Liste aproape sortate (unde se fac puține mutări).

2.4 Merge Sort (Sortarea prin Interclasare)

Context istoric: Este ideea matematicianului John von Neumann din 1945 [7]. El și-a dat seama că dacă spargi o problemă masivă în zeci de probleme mici, calculatorul o va rezolva mult mai repede și fără surprize neplăcute.

¹ENIAC a fost primul calculator digital electronic de uz general, ocupând o suprafață de aprox. 167 m² și folosind peste 17.000 de tuburi vidate [8].

Cum funcționează: Algoritmul taie lista nesortată pe jumătate. Apoi taie jumătățile în alte jumătăți, până rămân doar numere singure. Partea inteligentă abia acum începe: le lipește la loc (le interclasează), dar le pune direct în ordinea corectă.

Proprietate	Detalii
Complexitate teoretică	Timp: $O(n \log n)$ în toate cazurile. Memorie: $O(n)$.
Stabilitate	Stabil.
Scenariul ideal	Constant de rapid, indiferent de distribuția datelor.

2.5 Quick Sort (Sortarea Rapidă)

Context istoric: Inventat de Tony Hoare în 1959 [4]. Omul avea nevoie de o metodă prin care să sorteze rapid cuvintele dintr-un dicționar rusesc, ca să facă un program de traducere. Astăzi a ajuns cel mai folosit algoritm din lume tocmai pentru că se mișcă excelent.

Cum funcționează: Alege un număr oarecare din listă și îl numește „pivot”. Toate numerele mai mici ca el sunt aruncate în stânga, iar cele mai mari în dreapta. Apoi repetă procesul pentru grămada din stânga și cea din dreapta.

Proprietate	Detalii
Complexitate teoretică	Timp: $O(n \log n)$ (mediu), $O(n^2)$ (worst). Memorie: $O(\log n)$.
Stabilitate	Instabil.
Scenariul defavorabil	Liste sortate sau cu elemente identice (dacă pivotul e prost ales).

3 Arhitectura Proiectului

Ca să păstrez codul curat, am împărțit aplicația în C++ în trei module distincte:

- **Algorithm.h:** Aici am scris efectiv codul pentru cele 5 sortări.
- **Generators.h:** Modulul care îmi generează listele cu numere (aleatorii, sortate, inversate sau egale).
- **Benchmark.h:** Cronometrul proiectului. L-am construit folosind biblioteca oficială `<chrono>` [5] pentru a măsura timpul în milisecunde reale.
- **Functionarea in sine:** Programul, când este pornit, îți prezintă un meniu în care alegi ce dorești să testezi și să verifici.

4 Rezultate și Analiză Empirică

Pentru a vedea exact cum se comportă, am pus algoritmii să sorteze liste de câte **10.000 de elemente**. Datele strânse de mine în tabelul de mai jos arată cât de mare e prăpastia dintre ei.

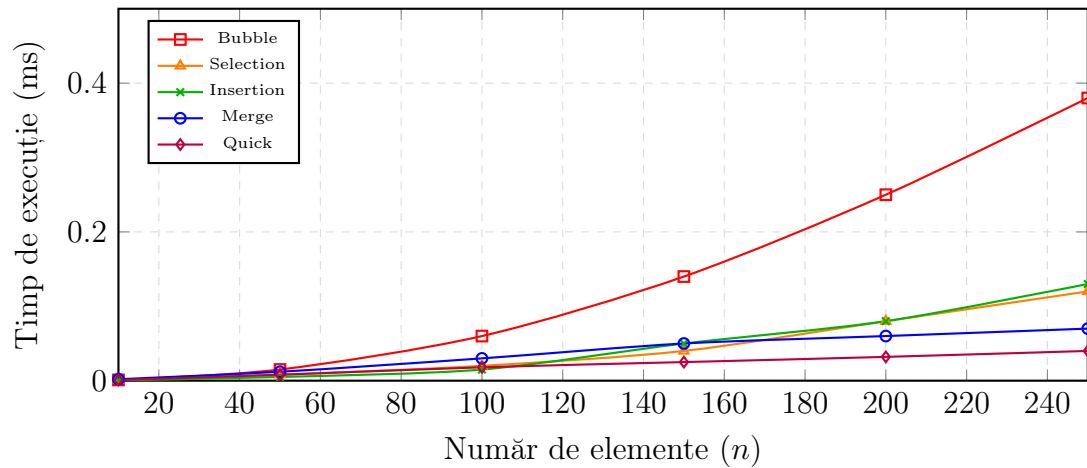
Distribuție	Bubble	Selection	Insertion	Merge	Quick
Random	657,44 ms	188,59 ms	213,15 ms	3,57 ms	1,89 ms
Sorted	0,03 ms	188,06 ms	0,08 ms	1,90 ms	0,56 ms
Reverse	749,53 ms	273,96 ms	544,90 ms	1,90 ms	0,33 ms
Flat (Egal)	514,31 ms	253,69 ms	181,93 ms	1,90 ms	47,39 ms

Tabela 1: Timpii de execuție per 10.000 de elemente obținuți în teste

5 Evaluarea Scalabilității: Analiză Grafică

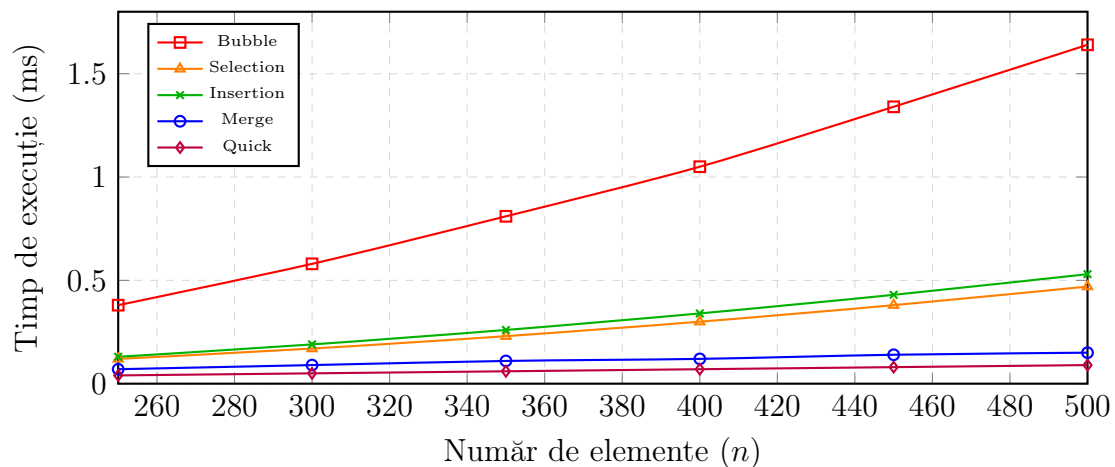
5.1 Fereastra Micro: 10 - 250 elemente

Aici m-a surprins cel mai mult Insertion Sort. La liste foarte mici e imbatabil, pentru că nu are funcții complicate care să îngreuneze procesorul.



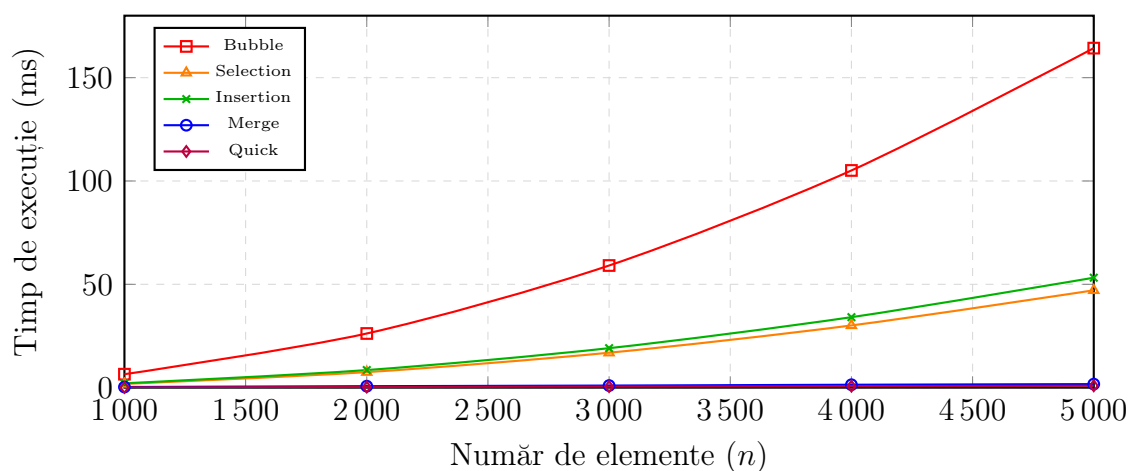
5.2 Fereastra Small: 250 - 500 elemente

Încă de la 300 de elemente, Bubble Sort începe să obosească vizibil. În tot acest timp, algoritmi modernii (Merge, Quick) abia dacă se încălzesc.



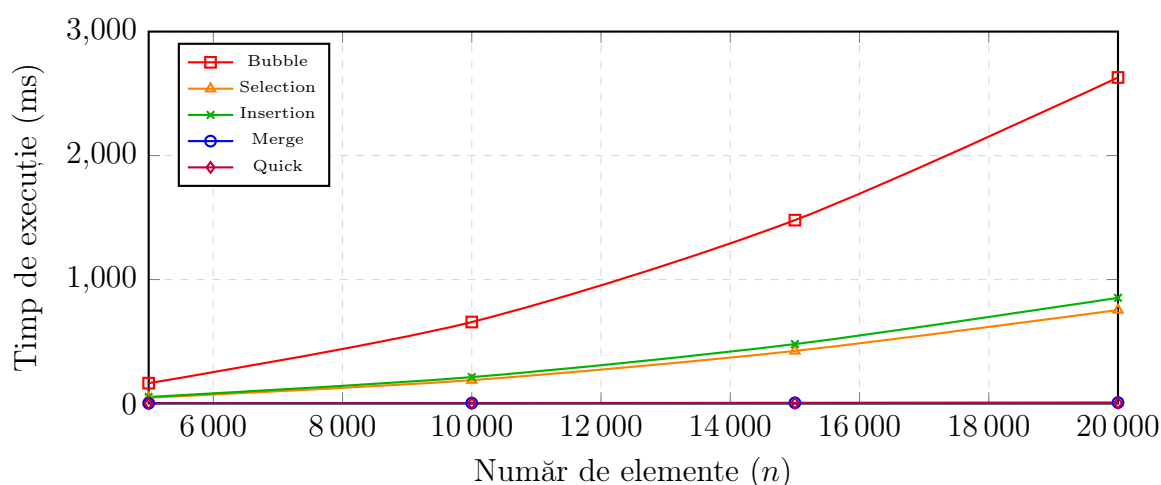
5.3 Fereastra Medium: 1.000 - 5.000 elemente

Aici e punctul de rupere. Graficul arată clar cum algoritmi vechi sar de 150 ms, devenind inutili dacă am vrea să îi folosim într-un program în timp real.



5.4 Fereastra Large: 5.000 - 20.000 elemente

La 20.000 de elemente, am stat să mă uit la ecran mai bine de 2,5 secunde ca Bubble Sort să termine de rulat. Pentru aplicațiile din ziua de azi, un astfel de timp de așteptare este pur și simplu inacceptabil.



6 Investigarea Anomaliilor de Performanță

Cel mai interesant moment din dezvoltarea acestui proiect a fost când am lovit două anomalii ciudate pe care nu le anticipasem inițial:

1. **Căderea Quick Sort-ului:** Am generat o listă plină cu același număr. Mă așteptam să o termine instant. În schimb, timpul i-a crescut brusc la 47,39 ms. De ce? Algoritmul meu își alegea prost pivotul și era forțat să facă mutări absolut inutile.

2. **Surpriza Insertion Sort:** Același algoritm vechi și aparent lent a reușit să obțină un timp șocant de bun (0,08 ms) pe o listă deja sortată. Practic, a bătut toți algoritmii complecși la un loc în acel scenariu specific.

7 Discuție Comparativă cu Literatura de Specialitate

Rezultatele obținute în cadrul acestui proiect sunt în concordanță cu concluziile prezentate de Singh și Kumar [3], în special în ceea ce privește comportamentul general al algoritmilor de sortare analizați.

În timp ce lucrarea academică menționată se concentrează pe o analiză teoretică, testele mele au scos în evidență anomalii ale implementării. Comparativ cu platformele precum Toptal [6], proiectul de față oferă date brute, permițându-mi să identific pragul de la care algoritmii de tip $O(n^2)$ devin inutilizabili în sisteme reale.

O diferență generală față de majoritatea lucrărilor care abordează acest subiect constă în faptul că, în cadrul acestora, este prezentat și codul sursă al algoritmilor analizați, oferind astfel o perspectivă mai practică asupra modului de implementare și funcționare al acestora.

8 Concluzii

Munca la acest proiect mi-a demonstrat că nu există cod perfect. Optimizarea începe mereu cu alegerea sculei potrivite.

Algoritmii de bază și poate că mai lenți sunt bine de știut, aceștia arătându-ne că o mică greșeală sau o perspectivă diferită pot influența enorm munca unei persoane și timpul de rezolvare. „Cei care nu învață din istorie sunt condamnați să o repete” (George Santayana).

Merge și Quick Sort sunt vitale pentru baze de date masive, dar algoritmii „clasici” rămân practici pentru seturi mici de date.

Bibliografie și Resurse Web

- [1] Knuth, D. E. (1998). *The Art of Computer Programming*. Addison-Wesley.
- [2] Cormen, T. H. et al. (2009). *Introduction to Algorithms*. MIT Press.
- [3] Singh, S., & Kumar, A. (2013). *A Comparative Study of Sorting Algorithms*. IJCA.
- [4] Hoare, C. A. R. (1961). *Algorithm 64: Quicksort*. ACM.
- [5] C++ Reference. *Header <chrono>*. <https://en.cppreference.com/w/cpp/header/chrono>
- [6] Toptal. *Sorting Algorithms Animations*. <https://www.toptal.com/developers/sorting-algorithms>
- [7] von Neumann, J. (1945). *Report on the EDVAC*.
- [8] Goldstine, H. H. (1972). *The Computer from Pascal to von Neumann*.
- [9] Friend, E. H. (1956). *Sorting on Electronic Computer Systems*. JACM.